

Trustworthy Smart Contract Repair for Tokenized Finance: Measurement Hazards, On Chain Evidence, and Governance Considerations

Joseph Taylor

Date:2026

Abstract

Tokenized finance is increasingly connecting blockchain based smart contracts with stablecoins, tokenized money market funds, securities, and other real world assets. This development creates opportunities for faster settlement, programmable financial services, fractional ownership, and improved transparency, but it also increases the importance of reliable smart contract security. Automated smart contract repair has emerged as a promising approach for identifying vulnerabilities and generating corrective patches. However, repair systems frequently depend on security analyzers whose measurements may contain false positives, false negatives, incomplete vulnerability coverage, inconsistent severity assessments, and duplicated findings. A repair mechanism that treats analyzer output as ground truth may therefore generate incorrect or unnecessary modifications. This problem becomes particularly significant in tokenized finance, where a flawed patch can affect asset balances, redemption mechanisms, collateralization, access control, and transaction settlement. This article proposes a trustworthy smart contract repair perspective that combines measurement hazard detection, on chain evidence, multi analyzer validation, patch verification, and governance controls. The methodology develops a conceptual framework for evaluating analyzer outputs against historical contract behavior and on chain transaction evidence. Four analytical tables are used to classify measurement hazards, evidence signals, repair controls, and governance requirements. The discussion demonstrates that trustworthy repair should not be defined solely by whether a vulnerability is detected or a patch compiles. Instead, repair quality should incorporate detection reliability, behavioral preservation, economic correctness, explainability, and governance accountability. The proposed approach provides a foundation for safer automated repair within stablecoin, tokenized money market fund, and real world asset ecosystems.

Keywords: smart contracts, automated repair, tokenized finance, stablecoins, real world assets, measurement hazards, on chain evidence, blockchain governance, smart contract security, trustworthy AI

Introduction

Blockchain based financial infrastructure has developed beyond cryptocurrency trading into a broader ecosystem involving stablecoins, decentralized finance, tokenized securities, tokenized funds, and real world assets. Tokenization allows financial or physical assets to be represented

digitally on distributed ledgers and can support programmable transactions, fractional ownership, automated settlement, and potentially greater operational efficiency. The Bank for International Settlements has identified tokenization as an important technological development for payments and securities markets while also emphasizing the associated operational, governance, and financial stability risks. ([Bank for International Settlements](#))

Smart contracts are central to many tokenized financial applications because they implement rules governing transfers, ownership, redemption, collateral, permissions, settlement, and other financial operations. Their programmability creates efficiency, but their deployment also creates security challenges. A coding error can become an economic vulnerability when the contract directly controls valuable digital assets. The 2026 OWASP Smart Contract Top 10 continues to identify access control, business logic, price oracle manipulation, flash loan facilitated attacks, input validation, unchecked external calls, arithmetic errors, reentrancy, and upgradeability among important smart contract risks. ([OWASP Foundation](#))

Automated repair has therefore attracted considerable research attention. Existing approaches employ template based repair, search based techniques, constraint solving, learning based approaches, and large language models. A recent systematic literature review identified 31 studies on automated smart contract repair and found that existing systems remain limited in the vulnerability classes they can address and that generated patches may not always prevent the targeted vulnerability. ([MDPI](#))

The challenge becomes more complicated when automated repair depends on security analyzers. Static analyzers, symbolic execution systems, fuzzers, and machine learning models do not provide perfect measurements. An analyzer can report a vulnerability that does not actually exist, fail to identify an existing vulnerability, duplicate findings, or assign an inappropriate severity level. Recent empirical research has demonstrated that analyzer performance can vary substantially across tools and datasets, reinforcing the importance of evaluating both false-positive and false-negative behavior. ([Springer Link](#))

These limitations introduce what can be described as **measurement hazards**. A measurement hazard occurs when an analyzer's output is treated as sufficiently reliable for automated decision making even though its uncertainty, coverage limitations, or contextual weaknesses have not been established.

In tokenized finance, the consequences of measurement hazards can be particularly serious. A repair that incorrectly changes access-control logic may prevent legitimate redemptions. A patch applied to an oracle-related vulnerability may unintentionally alter pricing behavior. Similarly, modifying a transfer or accounting function can affect the representation of tokenized assets. The Financial Stability Board has highlighted operational fragilities, oracle dependence, smart contract errors, governance weaknesses, and interconnectedness as important vulnerabilities associated with tokenization. ([Bank for International Settlements](#))

The objective of this article is therefore to develop a conceptual framework for **trustworthy smart contract repair for tokenized finance**. The framework combines analyzer measurement assessment, on chain evidence, multi-stage patch validation, and governance controls.

Methodology

This study adopts a conceptual, evidence driven methodology combining systematic literature synthesis, security analysis, and on chain evidence modeling. The approach is designed around the assumption that automated repair should be treated as a decision making pipeline rather than as a simple vulnerability detection and patch generation process.

The methodology consists of five stages: vulnerability measurement, measurement hazard assessment, on chain evidence collection, patch generation and validation, and governance approval.

Table 1. Measurement Hazards in Analyzer Based Smart Contract Repair

Measurement hazard	Description	Potential consequence	Recommended control
False positive	Analyzer reports a vulnerability that is not present	Unnecessary code modification	Multi analyzer confirmation
False negative	Existing vulnerability is not detected	Vulnerability remains after repair	Complementary analysis and testing
Duplicate findings	Same weakness reported multiple times	Incorrect risk prioritization	Finding normalization
Severity distortion	Incorrect severity assigned to finding	Poor repair prioritization	Context sensitive severity assessment
Limited coverage	Analyzer detects only selected vulnerability classes	Undetected security weaknesses	Analyzer ensemble
Context blindness	Tool ignores economic or application context	Incorrect patch	Semantic and financial-context analysis
Version sensitivity	Tool behaves differently across compiler or language versions	Unreliable results	Version controlled testing

The first stage involves collecting vulnerability measurements from multiple analyzers. Rather than accepting a single analyzer result as definitive, the framework compares outputs from

different analysis techniques. This approach is important because tools based on static analysis, symbolic execution, fuzzing, and machine learning have different strengths and weaknesses. Surveys of smart contract vulnerability detection demonstrate the broad diversity of available analytical approaches. ([ScienceDirect](#))

The second stage evaluates the reliability of each finding. A vulnerability score should therefore incorporate analyzer agreement, historical evidence, exploitability, code context, and behavioral evidence. For example, a reentrancy warning supported by several independent analyzers and confirmed by transaction traces should receive greater confidence than a warning produced by only one analyzer without supporting evidence.

The third stage introduces **on chain evidence**. Instead of examining source code independently from actual system behavior, the proposed methodology examines transaction histories, event logs, contract interactions, token transfers, gas patterns, administrative actions, and other observable blockchain information.

This is particularly relevant for tokenized finance because on chain activity can reveal how a contract actually behaves in production. Recent research on stablecoin transactions has demonstrated the value of analyzing large collections of blockchain event logs to understand transaction complexity and contract-based financial activity. ([Bank for International Settlements](#))

Table 2. On Chain Evidence Signals for Trustworthy Repair

Evidence signal	Example	Repair relevance
Transaction history	Repeated contract calls	Identifies normal behavioral patterns
Event logs	Transfer, mint, burn, redemption events	Establishes expected state transitions
Token flows	Movement between addresses	Detects abnormal asset movement
Gas consumption	Changes before and after repair	Detects unexpected computational effects
Administrative activity	Ownership or role changes	Validates access-control behavior
Oracle updates	Price feed changes	Helps evaluate oracle-dependent logic
Failed transactions	Reverted or unsuccessful calls	Identifies operational anomalies
Upgrade activity	Proxy implementation changes	Supports upgradeability analysis

The fourth stage involves patch generation. Candidate patches may be produced using predefined repair templates, program transformation techniques, constraint based methods, or AI assisted systems. However, patch generation should not automatically result in deployment. Every patch must pass behavioral and security validation.

The fifth stage involves governance approval. The repair process should record the original vulnerability, evidence supporting the finding, proposed modification, validation results, responsible reviewers, and deployment decision. This produces an auditable repair history.

Table 3. Trustworthy Repair Pipeline

Stage	Primary activity	Evidence required	Decision
1. Detection	Identify possible vulnerability	Analyzer findings	Candidate vulnerability
2. Measurement assessment	Evaluate analyzer reliability	Tool agreement and historical performance	Confidence score
3. Evidence validation	Compare with on chain behavior	Transactions and event logs	Confirmed or uncertain finding
4. Patch generation	Produce candidate repairs	Vulnerability context and repair rules	Candidate patch
5. Verification	Test security and behavior	Unit, fuzz, symbolic and regression tests	Valid or rejected patch
6. Governance review	Human and organizational assessment	Audit trail and risk assessment	Approve, modify, or reject
7. Deployment	Apply verified patch	Deployment controls	Controlled release
8. Monitoring	Observe post repair behavior	On chain metrics	Continue, rollback, or investigate

This methodology is consistent with broader trustworthy AI principles emphasizing validity, reliability, security, resilience, accountability, transparency, and explainability. NIST's AI Risk Management Framework specifically organizes risk management around governing, mapping, measuring, and managing risks, providing a useful conceptual basis for AI assisted smart contract repair. ([NIST](#))

Discussion

The central argument of this study is that smart contract repair should not be evaluated exclusively according to whether a generated patch removes a detected vulnerability. A trustworthy system

must establish whether the vulnerability measurement itself is reliable and whether the patch preserves legitimate financial behavior.

This distinction is important because automated repair creates two potential sources of error. The first is **detection error**, where the system incorrectly identifies or misses a vulnerability. The second is **repair error**, where the system correctly identifies a vulnerability but generates an inappropriate modification.

For tokenized financial systems, either type of error can have significant consequences. Consider a tokenized money market fund in which a smart contract controls subscription and redemption. If an analyzer incorrectly identifies a legitimate access-control mechanism as a vulnerability, an automated patch could weaken restrictions and expose the contract to unauthorized operations. Conversely, if the analyzer misses a genuine authorization weakness, attackers could potentially manipulate administrative functions.

Stablecoins present another important case. Stablecoin systems rely on smart contracts for minting, burning, transferring, and sometimes freezing tokens. Their financial significance extends beyond simple token transfers because stablecoins can interact with exchanges, lending platforms, collateral systems, and other financial contracts. Research has also shown that stablecoins can have important relationships with traditional financial markets and money market instruments. ([ScienceDirect](#))

Consequently, repair decisions should consider not only code-level security but also the economic function of the affected contract.

Measurement Hazards and Analyzer Reliability

The concept of measurement hazard is particularly important because security analyzers are often treated as objective sources of truth. In reality, an analyzer produces measurements according to a particular model, algorithm, dataset, configuration, and set of assumptions.

For example, a static analyzer may identify a potentially dangerous external call without understanding the economic conditions under which that call is made. Similarly, an automated system may identify a suspicious authorization pattern without understanding the legitimate administrative architecture of a tokenized asset platform.

Recent research has found substantial differences among smart contract security analyzers, including variations in accuracy, false-positive behavior, and execution time. ([Springer Link](#)) This suggests that analyzer output should be interpreted probabilistically rather than treated as unquestionable evidence.

A trustworthy repair framework should therefore maintain an **analyzer confidence profile**. Each analyzer can be evaluated according to historical precision, recall, vulnerability coverage, execution reliability, and performance across different contract types.

On Chain Evidence as a Validation Layer

On chain evidence provides an additional layer of validation because it captures actual system behavior. Transaction histories can reveal whether a suspected vulnerability corresponds to abnormal behavior, whether a function is routinely used, and whether a proposed repair changes important operational patterns.

This is particularly useful for tokenized real world assets. Tokenized securities, funds, real estate interests, and other assets may involve complex relationships between on chain tokens and off chain legal or financial claims. A code-level modification cannot be considered safe if it unintentionally changes the mechanisms through which ownership, redemption, transfer restrictions, or settlement are represented.

The BIS has noted that tokenization can improve efficiency and transparency but can also introduce new operational complexity, third-party dependencies, liquidity risks, and governance challenges. ([Bank for International Settlements](#)) Therefore, repair systems should explicitly account for dependencies such as price oracles, custodians, bridges, identity systems, and external data providers.

Governance Considerations

Technical verification alone is insufficient for high-value tokenized financial applications. Governance is required to determine who is authorized to initiate a repair, who validates the proposed patch, who approves deployment, and how emergency actions are documented.

Table 4. Governance Requirements for Trustworthy Smart Contract Repair

Governance dimension	Requirement	Purpose
Accountability	Assign responsibility for repair decisions	Prevents ambiguous ownership
Transparency	Maintain repair logs and evidence	Enables independent review
Explainability	Document why a patch was generated	Supports technical and regulatory review
Access control	Restrict repair deployment authority	Prevents unauthorized modifications
Independent review	Require secondary validation for critical patches	Reduces repair error
Version control	Preserve original and modified code	Enables auditability

Emergency procedures	Establish controlled emergency repair processes	Limits damage during active incidents
Post deployment monitoring	Continuously evaluate repaired contracts	Detects unintended consequences

Governance is particularly important when AI or machine learning is incorporated into repair. An AI system may generate a plausible patch without fully understanding the financial or legal implications of the modification. NIST emphasizes that trustworthy AI requires continuous consideration of risk throughout design, development, deployment, use, and evaluation. ([NIST](#))

This principle can be transferred to smart contract repair by requiring continuous monitoring rather than treating deployment as the end of the repair process.

Toward a Trustworthy Repair Architecture

The proposed framework can therefore be viewed as a layered architecture. The first layer is **measurement**, where multiple analyzers identify potential vulnerabilities. The second layer is **measurement hazard detection**, which evaluates disagreement, uncertainty, and analyzer limitations. The third layer is **on chain evidence**, which tests whether observed blockchain behavior supports the suspected vulnerability. The fourth layer is **repair validation**, where candidate patches undergo functional, security, and economic testing. The final layer is **governance and monitoring**, where human reviewers approve deployment and monitor the repaired system.

This approach is preferable to fully autonomous repair because it recognizes that security decisions in tokenized finance are socio-technical decisions. Code behavior, financial incentives, governance structures, legal arrangements, and user expectations can all influence whether a repair is appropriate.

Existing automated repair research supports the need for this cautious approach. The literature indicates that automated repair systems have made progress but remain constrained by vulnerability coverage and patch effectiveness. ([MDPI](#)) Research into LLM assisted repair also demonstrates that complex vulnerabilities, particularly access-control weaknesses, require contextual reasoning rather than simple pattern replacement. ([arXiv](#))

The proposed framework therefore does not reject automation. Instead, it positions automation within a controlled evidence and governance process.

Conclusion

Trustworthy smart contract repair is becoming increasingly important as blockchain infrastructure expands into tokenized financial markets. Stablecoins, tokenized funds, securities, and real world assets depend on smart contracts to perform functions that can directly affect ownership, settlement, redemption, pricing, and asset movement. As the economic importance of these

systems increases, the limitations of automated security analyzers and repair mechanisms become more consequential.

This article introduced a conceptual framework based on four central principles: **measurement hazard detection, on chain evidence, verified repair, and governance accountability**. The framework recognizes that analyzer outputs can contain false positives, false negatives, duplicated findings, incomplete coverage, and context-related errors. Consequently, a vulnerability report should be treated as evidence requiring validation rather than as an unquestionable fact.

On chain evidence can strengthen this validation by connecting code-level findings with actual transaction behavior. Multi analyzer comparison can further reduce dependence on a single measurement source. Candidate patches should then undergo security, functional, regression, and economic validation before deployment.

Finally, governance provides the accountability layer required for high-value financial infrastructure. Repair decisions should be documented, independently reviewed where appropriate, access-controlled, and monitored after deployment. The resulting approach shifts smart contract repair from simple automated patch generation toward **evidence-based, auditable, and continuously monitored security engineering**.

As tokenization develops, trustworthy repair mechanisms will become increasingly important not only for technical security but also for maintaining confidence in digital financial infrastructure. The integration of analyzer reliability, blockchain evidence, automated verification, and governance offers a practical direction for developing safer and more accountable smart contract repair systems.

References

1. Kiani, R., & Sheng, V. S. (2024). Automated Repair of Smart Contract Vulnerabilities: A Systematic Literature Review. *Electronics*, 13(19), 3942. <https://doi.org/10.3390/electronics13193942>. (MDPI)
2. Panda, S. K., Kuchipudi, J. D., Sahoo, A., DS, S., Manivannan, R., & Choudari, S. (2024, November). Prediction of Obesity in adolescent and childhood using machine learning models. In *2024 International Conference on Advances in Computing, Communication and Materials (ICACCM)* (pp. 1-6). IEEE.
3. Mohd, I. (2026, February). Evolving Legal Structures: A Study of Medical Device Governance in the Indian Context. In *2026 Contemporary Computing Innovations Conference (CCIC)* (pp. 1-7). IEEE.
4. Staley, I. (2026). Design and Adoption Signals in Tokenized Finance: Evidence from On-Chain Stable coin, MMF, and RWA Activity.

5. OWASP Foundation. (2026). *OWASP Smart Contract Top 10: 2026*. OWASP Smart Contract Security. ([OWASP Foundation](#))
6. National Institute of Standards and Technology. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1. ([NIST](#))
7. Bank for International Settlements. (2025). *The Next-Generation Monetary and Financial System*. BIS Annual Economic Report. ([Bank for International Settlements](#))
8. Bank for International Settlements. (2025). *Financial Stability Implications of Tokenisation: Executive Summary*. Financial Stability Institute. ([Bank for International Settlements](#))
9. Arveti, S., Hadkar, A., & Nutalapati, M. T. (2025). Cloud-Native Scalable Learning-to-Rank Architecture for Real-Time Search Optimization. *International Journal of Computational and Experimental Science and Engineering*, 11(4). <https://doi.org/10.22399/ijcesen.5118>
10. Al-Subaey, F. S. (2026). Ethical leadership and organisational citizenship behaviour: the mediating role of employee commitment and the moderating role of Islamic work ethic. *Frontiers in Psychology*, 17, 1847356.
11. Bank for International Settlements. (2025). *Cryptocurrencies and Decentralised Finance: Functions and Financial Stability Implications*. BIS Papers No. 156. ([Bank for International Settlements](#))
12. Ahmed, R., & Aldasoro, I. (2025/2026). *Stablecoins and Safe Asset Prices*. BIS Working Papers No. 1270. ([Bank for International Settlements](#))
13. Bank for International Settlements. (2026). *The Anatomy of Stablecoin Transactions*. BIS Working Papers. ([Bank for International Settlements](#))